

<변수 선언>

```
1  `include "opcodes.v"
2
3  module cpu (readM, writeM, address, data, ackOutput, inputReady, reset_n, clk);
4      output reg readM; // "read" signal to memory
5      output reg writeM; // "write" signal to memory
6      output reg [`WORD_SIZE-1:0] address; // target memory address
7      inout [`WORD_SIZE-1:0] data; // data for reading or writing (can be used as both input and output)
8      input ackOutput; // signal from memory ("data is written")
9      input inputReady; // signal from memory ("data is ready for reading")
10     input reset_n; // reset your CPU (registers, PC, etc)
11     input clk; // clock signal
12
13     reg [`WORD_SIZE-1:0] pc; // program counter
14
15     reg [3:0] Op_Code; // operation code
16     reg [1:0] rs, rt, rd, wr; // register index
17     reg [`WORD_SIZE-1:0] regs[3:0]; // register file
18     reg [5:0] Func_Code; // function code
19     reg [7:0] Immediate; // immediate value
20     reg [`WORD_SIZE-1:0] Sign_Extend_Immediate; // sign extension
21     reg [11:0] Target_Address; // jump address
22
23     reg [`WORD_SIZE-1:0] A, B; // ALU operand
24     wire [`WORD_SIZE-1:0] C; // ALU result
25     wire bcond; // ALU branch condition
26
27     reg load_check; // check if instruction was 'LWD_OP' or not
28
29     reg Register_Write_Condition; // write condition boolean true or false
30     reg [`WORD_SIZE-1:0] Write_Back_Register; // write register value
31
32     // If store, put register value of reg[rt] in memory, otherwise read data
33     assign data = (writeM == 1) ? regs[rt] : `WORD_SIZE'bz;
```

#Op_Code / func_Code를 두어서, 해당 값을 ALU로 보낸다. 그러면 CPU 내부에서 Op_Code가 15인 경우에는, func_Code로 Instruction을 구분하여 알맞은 동작을 한 뒤 output layer C로 값을 내보낸다. (그 값은 Write_back_register로 임시 저장됨)

#Regs / 실제 변수가 저장되는 레지스터 4개인 이유는, tb코드에 레지스터 인덱스가 0부터 3까지 4개가 있었다.

#Immediate, Sign_Extend_Immediate / Instruction에서 받은 offset 값이 Immediate에 들어가고, 만약 I-type Instruction을 만났다면, Sign_Extend_Immediate을 해준다.

Immediate와 Sign_Extend_Immediate를 따로 둔 이유는 원래 그냥 shift를 해주려고 했는데 안 되어서 보니까 8bit에서 shift가 될 뿐 16bit로 확장이 안 되더라, 그래서 새로운 변수를 넣었다.

#load_check는, 다른 것들은 저희가 만들어놓은 틀에서, clk 내부에서 한번 사이클이 돌면 되는데

load instruction의 경우 두 번의 사이클이 돌아야해서, 이 경우 readM을 한 번 더 읽는데, 하는 동작을 다르게 해주려고 새로운 변수를 하나 넣었다.

#Write Register_Write_Condition은 말 그대로 계산한 값을 레지스터에 넣어나 하는 상황이나, 아니냐를 구분해주는거고

#WriteM은 data를 tb코드에 존재하는 memory에다가 address를 보고 직접 써주는건데, 현재 읽어들인 값을 보고, 만약 opcode가 SWD에 해당하면, \$rs + offset를 address에 넣어서 데이터를 써줄 memory address를 구하고, writeM 을 1로 만들어줘서 tb코드에서 값을 쓰도록 했습니다.

data를 input으로 쓰느냐 / output으로 쓰느냐도 writeM의 값에 따라 설정해주었고,

tb코드를 보니까 현재 address에 해당하는 memory에 값이 쓰여진 뒤, delay가 있고 ackOutput의 값이 변하길래, ackOutput 값이 변하면 write가 된거니까 writeM을 다시 꺼줬습니다.

```
98
99
100    //Data is written in memory -> set writeM back to 0
101    always @(posedge ackOutput) begin
102        writeM <= 0;
103    end
104
105
```

이 부분 그냥 지워버려도 어차피 clk negedge 되면 writeM 알아서 0 되기 때문에 코드가 들어감. 굳이 써준 이유는 데이터가 이 전에 이미 옮겨진다는 것을 명시적으로 나타내기 위해서 작성했다.

// 55번째 줄 / posedge inputReady

```
53
54     //Read data from memory
55     always @(posedge inputReady) begin
56         if(load_check == 0)begin //instruction fetch
57             Op_Code <= data[`WORD_SIZE-1:12];
58             Target_Address <= data[11:0];
59             rs <= data[11:10];
60             rt <= data[9:8];
61             rd <= data[7:6];
62             Func_Code <= data[5:0];
63             Immediate <= data[7:0];
64             readM <= 0;
65         end
66
67         else if(load_check == 1) begin //put memory value in register
68             wr <= rt;
69             Write_Back_Register <= data;
70             readM <= 0;
71             Register_Write_Condition <= 1;
72         end
73     end
74
```

readM이 1이 되면, 값을 읽고나서 inputReady가 1이 되니까, 이 때 읽어온 data 값을 pdf에 있는 instruction format에 맞추어 읽어주도록 했다.

#load function 이 부분에서 어려웠던 것은 load function의 경우, 일단 처음에(load_check == 0 일때) data에서 읽은 \$rs와 offset을 가지고, 새로운 address를 계산하고, 그리고 그 address register에 있는 값을 목적지 register(\$rt)에 넣어줘야 했다.

Clk이 posedge가 되면 값을 또 새롭게 읽어들일테니까, 그 전에, 즉, 이 것이 하나의 clk에서 이루어져야 한다고 생각했다.

새로운 변수 load_check를 넣어서 readM과 load_check를 동시에 1로 만들어주고(이러면 tb에서 address 값을 가지고 data를 다시 읽음), case를 나눠서 load_check가 1이면, 다시 읽어들이

loaded_data를 Write_Back_Register에 넣도록 계산했다. (이거 없애면 코드 성공 안됨)

151번째 줄 // posedge clk

```
//Clock
always @(posedge clk) begin
    if(!reset_n) begin//If reset
        readM <= 0;
        writeM <= 0;
        address <= 0;
        pc <= 0;
        A <= 0;
        B <= 0;
        Register_Write_Condition = 0;
        Op_Code <= 0;
        load_check <=0;
    end
    else begin//read current instruction
        readM <= 1;
        Register_Write_Condition <= 0;
        writeM <= 0;
        address <= pc;
        load_check <= 0;
    end

    if(Register_Write_Condition) begin//write back register
        regs[wr] <= Write_Back_Register;
    end
end

endmodule
```

Handwritten notes: A blue circle around "Register_Write_Condition = 0;" with an arrow pointing to the right. Below it, the text "Non-blocking" is written in blue. To the right of the first circle, there is a blue diagram showing a signal path with a delay element (a circle with a diagonal line) and an arrow pointing to the right.

#1 Register_Write_condition = 0; 을 Register_Write_condition <= 0 로 수정하여야 함. (이 코드에서는 load뒤 reset하는 작업이 없어서 fail이 안뜨지만, 원래는 #2와 같은 이유로 잘못된 코드임 #2가 잘못됐다는게 아니라, #2도 '='로 바뀌면 잘못된 코드가 됨)

#2 else begin에 Register_Write_Condition <= 0; 이 있음에도 if 문이 돌아가서 regs[wr]에 값이 써지는 이유는, non-blocking으로 선언되었기 때문이다.

기존의 Register_Write_Condition이 넘어올 때 '1'의 값을 가지고 넘어왔으므로, 이 값을 이용해 if 문이 돌아가고, Register_Write_Condition은 '0'이 되어 다음으로 넘어간다.

(만약 <= 를 = 로 바꿔주면 성공 1 / 실패 15가 된다(값이 입력되지 않음))